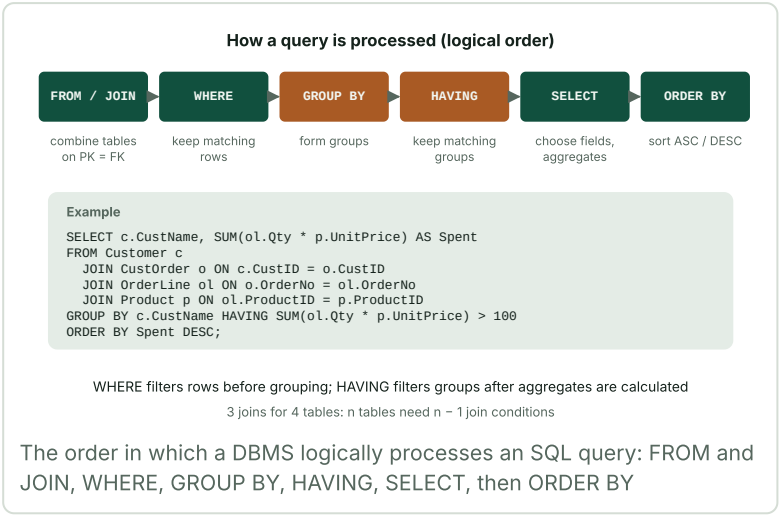
# SQL queries: SELECT, aggregates, grouping and joins

IS06 FACT FILE

**Specification: students should be able to**

- Write SQL SELECT queries using WHERE, ORDER BY, AND, OR, LIKE and BETWEEN
- Use aggregate functions with GROUP BY
- Write queries that join two or three tables

**Key words:** SELECT, FROM, WHERE, ORDER BY, LIKE, wildcard, BETWEEN, aggregate function, GROUP BY, HAVING, INNER JOIN, alias




The order in which a DBMS logically processes an SQL query: FROM and JOIN, WHERE, GROUP BY, HAVING, SELECT, then ORDER BY

## Single-table queries

A SELECT query names the fields to display, the table in FROM, an optional WHERE condition and an optional ORDER BY. Conditions use comparison operators and combine with AND, OR and NOT, with brackets controlling the order. LIKE matches patterns using % for any sequence of characters and _ for a single character. BETWEEN is inclusive of both limits, and IN checks whether a value is in a list.

## Aggregates and grouping

Aggregate functions calculate one value from many rows: COUNT, SUM, AVG, MIN and MAX. Used alone they summarise the whole result; with GROUP BY they produce one row per group. Any field shown that is not inside an aggregate must be listed in GROUP BY. WHERE filters individual rows before grouping, while HAVING filters the groups after the aggregates have been calculated. AS gives a calculated column a readable name.

## Joins

Because normalised data is split across tables, queries often need joins. An inner join returns only rows where the foreign key in one table matches the primary key in the other. Joining three tables needs two join conditions, following the path of relationships in the ER diagram, for example Customer to CustOrder to OrderLine. If a join condition is omitted, the result is a Cartesian product in which every row of one table is paired with every row of the other.

## T Key terms from the mark schemes

**Aggregate function**
An SQL function that calculates a single value from a set of rows, such as COUNT, SUM, AVG, MIN or MAX.

**GROUP BY**
An SQL clause that groups rows with the same value in a field so aggregate functions are calculated for each group.

**HAVING**
An SQL clause that filters groups after aggregation, in the way WHERE filters individual rows.

**Join**
An operation that combines rows from two or more tables by matching a foreign key with the related primary key.

**Wildcard**
A symbol used with LIKE to match unknown characters: % for any sequence and _ for exactly one character.

### Exam focus

Read the question carefully for exactly which fields to display and in what order to sort. Marks are usually awarded per clause: SELECT fields (1), FROM tables (1), join condition (1), each criterion (1), ORDER BY (1). Use the table and field names given, quote text, and write dates in the format shown. When asked to 'state the output' of a query on a given table, list the rows in the order the query would return them, with only the selected fields.

## ? Check yourself

1. Write an SQL query to list the Surname and Forename of all pupils in YearGroup 14, in alphabetical order of Surname. [3]
2. Write an SQL query to display all products whose name begins with 'Organic' and whose price is between £1 and £3. [3]
3. Explain the difference between WHERE and HAVING. [2]

**Answers:** 1. Model answer:
SELECT Surname, Forename
FROM Pupil
WHERE YearGroup = 14
ORDER BY Surname;
Marks: SELECT Surname, Forename FROM Pupil (1) / WHERE YearGroup = 14 (1) / ORDER BY Surname (ASC) (1). Accept any sensible table name.  2. Model answer:
SELECT *
FROM Product
WHERE ProductName LIKE 'Organic%'
 AND UnitPrice BETWEEN 1 AND 3;
Marks: SELECT * FROM Product (1) / ProductName LIKE 'Organic%' (accept * wildcard in Access) (1) / AND UnitPrice BETWEEN 1 AND 3 (accept >= 1 AND <= 3) (1).  3. WHERE filters individual rows before they are grouped / aggregated (1) / HAVING filters groups after GROUP BY and aggregation, so it can use aggregate functions such as COUNT(*) (1).